중간발표 대본 — 지도 기반 노선 타당성 분석 시뮬레이터 (RailFeas)

컴퓨터시스템공학과 3학년 2학기 · 202245079 신선호 · 심화응용프로젝트

발표 자료「RailFeas 중간 발표」15장의 발표 메모를 장 순서대로 옮겼습니다.

읽는 법

검은 글씨는 꼭 말할 부분, 회색 글씨(생략 가능)는 시간이 모자랄 때 빼도 흐름이 끊기지 않는 문단입니다. 같은 이야기를 다른 장에서도 하거나, 계수를 하나씩 풀어 읽거나, 화면에 이미 적힌 사례를 자세히 드는 문단입니다. 시간은 1분에 300자(공백 제외)로 잡았습니다 — 실제로 한 번 소리 내어 읽어 보고 맞추세요.

전문 약 29분 · 회색을 빼면 약 21분

장	내용	전문(분)	줄이면(분)
1	RailFeas	0.8	0.8
2	문제	0.7	0.7
3	사용 흐름	0.7	0.6
4	실제 화면	2.0	1.7
5	시스템 구조	1.0	0.5
6	기술 스택	1.1	0.6
7	외부 자료	3.1	1.1
8	데이터 파이프라인	0.9	0.7
9	비용 모델	2.8	1.9
10	수요 모델	2.0	1.3
11	B/C 산출	2.6	1.8
12	추천 알고리즘	1.7	1.1
13	정차역 자동 배치	2.3	1.8
14	검증	2.9	2.0
15	현재 상태	4.7	4.1
	합계	**29.4**	**20.7**

1. RailFeas — 표지

전문 0.8분 · 줄이면 0.8분

안녕하십니까, 컴퓨터시스템공학과 신선호입니다. 제가 만든 RailFeas 를 발표하겠습니다.

한 문장으로 말씀드리면, 지도에 선을 그으면 그 노선을 지을 만한지 숫자로 답해 주는 웹 서비스입니다.

중요한 점은 숫자를 지어내지 않았다는 것입니다. 건설비는 실제 철도사업 38건, 수요는 수도권 전철 514개 역에서 회귀로 얻었고, B/C 원단위는 국토교통부 투자평가지침 제7판을 그대로 따랐습니다.

오늘은 기술 스택, 사용한 API, 비용과 수요 모델, B/C 산출, 그리고 수단 추천 알고리즘 순서로 말씀드리겠습니다.

2. 문제 — "이 노선, 지을 만한가"에 답하려면 지금은 몇 달이 걸립니다

전문 0.7분 · 줄이면 0.7분

먼저 왜 만들었는지 말씀드리겠습니다.

예비타당성조사는 몇 달이 걸립니다. 그런데 후보 노선을 추리는 초기 단계에서 정말 필요한 것은 정확한 한 숫자가 아니라, 후보들 중 어느 것이 나은가 하는 순위입니다.

그래서 입력은 지도 위의 선 하나로 최대한 줄였고, 출력은 수단 열 개의 B/C 비교표로 냅니다.

오른쪽 어두운 칸이 중요합니다. 제 모델의 오차는 비용 1.25배, 수요 1.75배입니다. 이걸 숨기지 않고 화면에 그대로 적었습니다. 절대값을 믿으라는 서비스가 아니라, 순위를 보라는 서비스입니다.

3. 사용 흐름 — 클릭 두 번에서 비교표까지

전문 0.7분 · 줄이면 0.6분

쓰는 흐름은 네 단계입니다.

지도를 두 번 클릭하면 연장이 바로 나옵니다. 저장하고 비용 비교하기를 누르면 수단 열 개를 한꺼번에 계산하고, 정차역도 수단별 역간격에 맞춰 자동으로 놓습니다.

아래 칸을 강조하고 싶습니다. 신설안끼리만 비교하면, 이미 길이 있는 곳에 또 짓는 안이 1등으로 올라와도 보이지 않습니다. 그래서 기존 도시철도와 KTX를 찾아서 같은 표 안에 한 줄로 넣었습니다. 추가 건설비가 0인 대안이니까요.

> 생략 가능 · 실제로 서울에서 부산을 그으면 KTX 경부선이 비교표에 뜨고, 지도에는 청록 점선으로 그려집니다.

4. 실제 화면 — 시흥~안양 14km 를 실제로 돌린 결과입니다

전문 2.0분 · 줄이면 1.7분

말씀드린 것이 실제로 돌아가는 화면입니다.

왼쪽은 시흥에서 안양까지 14킬로미터를 그은 것입니다. 지도의 파란 점이 자동으로 놓인 정차역이고, 연성동 능곡동 목감동 박달동처럼 이름까지 붙습니다. 제가 역 이름을 적은 게 아니라, 역을 놓은 자리에서 가장 가까운 행정동 이름을 가져온 것입니다.

오른쪽 표에 수단 열 개가 B/C 순으로 서 있습니다. 1등이 BRT 고급형 지상, B/C 2.05 이고 건설비 629억입니다. 지하철 지하는 1조 9216억에 B/C 0.15 입니다. 같은 노선인데 수단에 따라 건설비가 30배, B/C가 14배 갈립니다. 이 표를 보여 주는 게 이 서비스의 목적입니다.

> 생략 가능 · 분홍색으로 깔린 줄들은 수송능력을 넘거나 수요 하한에 못 미쳐 경고가 붙은 안이고, 맨 아래 BRT 저급형은 B/C 가 0 이라 흐리게 찍힙니다. 계산 실패가 아니라 시내버스보다 느려서 절감할 시간이 없다는 결과입니다.

오른쪽 화면은 노선끼리 견주는 자리입니다. 세 노선을 나란히 놓았는데, 이미 있는 수단 행을 보시면 서울~부산에만 기존 KTX 가 뜹니다. 추천안 B/C 가 2.05, 0.07, 0.01 입니다. 서울~부산은 처음에 1.67 이었는데, 이용객 한 명이 노선 끝까지가 아니라 평균 13.6킬로미터만 탄다고 고치면서 0.07 이 됐습니다. 318킬로미터 노선이라 영향이 가장 컸고, 14킬로미터인 시흥~안양은 거의 그대로입니다. 광명-화성이 0.01 인 것은 주변 인구가 2,085명뿐이라 그렇습니다.

두 화면 모두 꾸민 그림이 아니라 실제로 계산한 결과입니다.

5. 시스템 구조 — 세 덩어리로 나뉩니다

전문 1.0분 · 줄이면 0.5분

구조는 세 덩어리입니다.

브라우저는 React 와 MapLibre 로 지도를 그립니다. 백엔드는 Spring Boot 가 계산을 맡습니다. 저장소는 MySQL 과 Redis 입니다.

아래 파란 띠가 제가 제일 공들인 부분입니다. Python ETL 은 서비스가 돌 때 쓰이지 않습니다. 미리 원자료를 받아 회귀를 돌리고, 그 결과를 기준값 SQL 로 만들어 DB 에 넣어 둡니다.

> 생략 가능 · 이렇게 한 이유는 두 가지입니다. 하나는 요청마다 회귀를 돌리면 느리기 때문이고, 다른 하나는 관리자가 기준값을 화면에서 고칠 수 있어야 하기 때문입니다.

> 생략 가능 · 또 하나, 스키마는 JPA 가 만들지 않습니다. ddl-auto 를 validate 로 고정해서, SQL 파일이 만든 스키마와 엔티티가 어긋나면 아예 기동이 안 되게 했습니다.

6. 기술 스택 — 무엇을 왜 썼는가

전문 1.1분 · 줄이면 0.6분

기술 스택입니다.

> 생략 가능 · 프론트는 React 와 Vite 입니다. Vite 를 고른 이유가 하나 더 있는데, VWorld 지도 타일은 브라우저가 직접 부르면 인증키 도메인이 안 맞아 403 이 납니다. Vite 프록시가 대신 받아오면서 Referer 를 맞춰 줍니다. 키가 URL 에 노출되지 않는 효과도 있습니다.

백엔드는 Spring Boot 입니다. 인증은 JWT HS512 를 쓰고 구글·카카오 소셜 로그인을 붙였습니다.

한 가지 설명드리고 싶은 줄은 영속 행입니다. ddl-auto 를 validate 로 고정했습니다. JPA 가 테이블을 만들게 두면 SQL 파일과 엔티티가 조용히 어긋납니다. validate 면 어긋날 때 기동이 아예 안 되니까 바로 알 수 있습니다.

> 생략 가능 · 맨 아래, 상태 관리 라이브러리는 일부러 안 썼습니다. 화면이 하나라 React 상태로 충분했습니다.

7. 외부 자료 — 공공 API 일곱 곳에서 받았습니다

전문 3.1분 · 줄이면 1.1분

자료는 공공 API 일곱 곳에서 받았습니다.

수요 모델은 서울 열린데이터광장의 승하차 자료로 학습했고, 인구는 통계청 SGIS 집계구를 씁니다. 비용 회귀의 표본은 국가철도공단 철도건설현황입니다.

> 생략 가능 · 여섯 번째 줄이 재미있는데, 처음에는 수도권 집계구 5만 개만 모았습니다. 그러니까 지방에 노선을 그으면 인구 자료가 없어서 B/C 가 안 나왔습니다. OSM 에서 지방 역 좌표를 받아 수집 범위를 7만 개로 넓혀 해결했습니다.

아래 붉은 칸이 이번에 가장 많이 배운 부분입니다. 한국철도공사 API 가 403 이라 KTX 소요시간을 계속 비워 뒀는데, 국토교통부 TAGO 열차정보로 받아서 채웠습니다. 서울에서 부산이 44편 중앙 164분이고, 경로가 다른 11편은 중앙 198분입니다. 화면의 신설안이 모두 분 단위라 같은 단위로 적습니다.

그런데 받아서 끝이 아니었습니다. 세 번 데였습니다.

생략 가능 · 첫째, 열차 등급을 KTX 00번으로 걸고 받았습니다. 그런데 KTX 계열이 여섯 가지였습니다. 산천, 이음, 청룡이 다 다른 코드입니다. 그래서 강릉선이 통째로 비었습니다. 서울에서 진부 가는 열차가 전부 KTX-이음이었던 겁니다. 지금은 등급을 걸지 않고 전부 받아서 이름으로 거릅니다.

생략 가능 · 둘째, 우리 역 파일은 울산 괄호 통도사라고 적는데 TAGO 는 그냥 울산이라고 씁니다. 이름이 안 맞아서 울산이 빠졌습니다. 그래서 괄호를 벗겨 보는데, 벗기는 순서가 중요합니다. TAGO 에는 판교 괄호 경기와 판교 괄호 충남이 따로 있습니다. 괄호를 먼저 벗기면 엉뚱한 역의 시각표를 받습니다. 정확한 이름을 먼저 보고, 없을 때만 벗깁니다.

생략 가능 · 셋째가 가장 뜨끔했습니다. 운임이 크게 벌어지는 자리에서 계통을 갈랐는데, 밀양에서 천안아산이 1편 107분과 7편 100에서 115분으로 갈렸습니다. 2분 차이를 계통이라 부른 겁니다. 더 나쁜 건, 그 1편이 화면의 대표값이 된다는 점입니다. 계통을 가른 목적이 바로 한 열차의 시각을 대표값으로 쓰지 않는 것이었는데, 제 규칙이 그 일을 하고 있었습니다.

생략 가능 · 그래서 조건을 두 개 더 걸었습니다. 양쪽에 3편 이상 있고, 시간 범위가 겹치지 않을 때만 가릅니다. 자료로 가려 보니 둘 다 넘는 구간은 경부축 13개뿐이었습니다.

여기서 배운 건, 수집이 끝나면 몇 개 받았나가 아니라 뭐가 안 왔나를 세야 한다는 겁니다. 세 번 다 예외가 안 났고 로그도 조용했습니다. 지금은 노선별로 받은 구간 수를 찍어서, 노선 하나가 통째로 비면 바로 보이게 했습니다.

8. 데이터 파이프라인 — 원자료가 기준값이 되기까지

전문 0.9분 · 줄이면 0.7분

파이프라인은 네 단계입니다. 수집, 정제, 회귀, 적재입니다.

두 번째 칸을 보시면, 이상치를 지우지 않고 표시만 합니다. 지워 버리면 왜 뺐는지 나중에 설명할 수 없기 때문입니다. 그리고 추정치는 회귀 학습에서 뺍니다. 추정으로 만든 값으로 다시 추정하면 순환 오류가 생깁니다.

생략 가능 · 네 번째 칸, 생성된 SQL 에 생성 시각을 넣지 않았습니다. 넣으면 다시 돌릴 때마다 파일이 달라져서 진짜 변경이 묻힙니다.

아래 숫자들입니다. 철도사업 표본은 381건을 모았지만 회귀에는 A등급 38건만 씁니다. 사업비가 공시된 것만 쓴다는 뜻입니다. 표본이 적은 것은 분명한 한계이고, 뒤에서 다시 말씀드리겠습니다.

9. 비용 모델 — 실제 철도사업 38건에서 뽑았습니다

전문 2.8분 · 줄이면 1.9분

비용 모델입니다.

국가철도공단 철도건설현황에서 사업비가 공시된 38건을 골라 로그-로그 회귀를 돌렸습니다.
결정계수는 0.887, 교차검증 오차는 1.25배입니다.

생략 가능 · 식의 숫자가 어떤 역할인지 말씀드리겠습니다. 양변에 로그를 씌웠기 때문에 계수가 탄력성이 됩니다. 몇 퍼센트 늘면 몇 퍼센트 는다는 뜻입니다.

연장 계수 0.663 은 연장이 두 배가 되면 사업비는 1.58배만 는다는 뜻입니다. 두 배가 아니라 1.58배입니다. 이게 규모의 경제이고, 긴 노선일수록 킬로미터당 단가가 싸지는 이유입니다.

생략 가능 · 지하비율 0.515 는 로그를 안 씌운 비율이라 읽는 법이 다릅니다. 전 구간을 지하로 지으면 1.67배가 됩니다. 계수 중 가장 큽니다.

이 지하비율은 공식 자료가 없어 원래 40건 모두 추정치였습니다. 그중 17건을 OpenStreetMap 선로에 붙은 터널 표시로 직접 쟀습니다. 0과 1은 대개 맞았는데 중간값이 틀렸습니다. 용인경전철을 0.2 로 적어 뒀는데, 재 보니 선로가 거의 전부 고가라 0이었습니다. 실측으로 바꾸니 지하 할증이 1.78배에서 1.67배로 줄었고 오차는 그대로였습니다. 남은 23건은 아직 공사 중이라 잴 노선이 없어 추정치로 남겨 뒀습니다.

생략 가능 · 역수 0.257 은 역이 두 배면 1.20배입니다. 6.910 은 절편인데 단위를 맞추는 값이라 따로 뜻을 읽지 않습니다.

생략 가능 · 왼쪽 칸입니다. 회귀식을 그대로 서비스에 넣지 않았습니다. 격자에서 직선 근사해서 고정비 더하기 연장 곱하기 단가 꼴로 바꿨습니다. 이유는 관리자가 단가를 화면에서 고칠 수 있어야 하기 때문입니다. 로그 식이면 계수를 손으로 만질 수가 없습니다.

오른쪽 칸은 제가 실제로 겪은 문제입니다. 역수를 선형으로 넣었더니 계수가 음수로 나왔습니다. 역이 많을수록 싸진다는 뜻이 되는데 말이 안 되죠. 연장과 역수가 서로 묶여 있어서 그렇습니다. 로그로 바꾸니 분리됐습니다.

아래 회색 칸은 제가 가장 배운 부분입니다. 급행을 따로 가를까 검정했더니 t 값이 0.08 로 작았습니다. 보통 여기서 "차이 없음"이라고 쓰는데, 표본이 4건이라 검출 한계가 1.5배였습니다. 즉 1.5배 차이가 나도 못 잡아내는 검정이었습니다. 그래서 "차이가 없다"가 아니라 "못 본다"라고 적었습니다.

10. 수요 모델 — 수도권 전철 514개 역이 가르쳐 준 것

전문 2.0분 · 줄이면 1.3분

수요 모델입니다.

수도권 전철 514개 역의 승하차 자료로 회귀를 돌렸습니다. 설명 변수는 인구, 종사자, 환승 노선 수, 도심까지 거리 네 개입니다.

여기도 계수가 탄력성입니다. 환승 노선 계수 1.080 은 환승 노선이 두 배면 이용객이 2.11배가 된다는 뜻이고, 네 변수 중 기여가 가장 큽니다. 인구 0.437 은 인구가 두 배여도 이용객은 1.35배밖에 안 는다는 뜻입니다. 사람 수보다 어디에 붙어 있느냐가 더 중요하다는 말입니다.

생략 가능 · 도심거리 마이너스 0.012 만 로그를 안 씌웠습니다. 도심에서 10킬로미터 멀어지면 이용객이 11퍼센트 줄어듭니다. 부호가 마이너스인 것도 상식과 맞습니다.

왼쪽 칸. 처음에는 인구만 넣었는데 결정계수가 0.32 밖에 안 됐습니다. 서울역처럼 사는 사람은 적은데 낮에 붐비는 역을 10분의 1로 과소추정했습니다. 그래서 종사자 수를 주간인구 대리 변수로 넣고, 환승 노선 수를 넣어 0.551 까지 올렸습니다.

생략 가능 · 가운데와 오른쪽은 제가 실제로 만든 버그입니다. 환승 노선을 역 이름으로 묶었더니 서울시청과 부산 시청이 한 역이 됐습니다. 그리고 도심 거리를 서울시청으로 고정했더니 부산 역이 326km 떨어진 걸로 계산돼서 이용객이 2천 명으로 깎였습니다. 숫자는 나오는데 뜻이 없는 값이었습니다. 지금은 가장 가까운 광역시 도심을 기준으로 합니다.

아래 칸이 중요합니다. 오차 1.75배는 큽니다. 그런데 수단을 비교할 때는 같은 수요를 모든 수단에 똑같이 넣기 때문에, 어느 수단이 낫느냐 하는 결론은 바뀌지 않습니다. 바뀌는 건 B/C 의 절대값입니다.

11. B/C 산출 — 원단위는 지어내지 않고 지침에서 가져왔습니다

전문 2.6분 · 줄이면 1.8분

B/C 산출입니다.

B/C 는 개통 후 40년 편익의 현재가치를 공사 5년 사업비의 현재가치로 나눈 값입니다. 여기서 중요한 건 원단위를 제가 정하지 않았다는 점입니다. 전부 국토교통부 투자평가지침 제7판에서 가져왔고, 표 번호까지 적어 뒀습니다.

표의 마지막 줄, 전환 전 수단은 시내버스 하나가 아닙니다. 2호선을 따라 지하철을 그리면 그 사람들은 원래 2호선을 타던 사람들이라, 시내버스보다 빨라진 만큼이 아니라 2호선보다 빨라진 만큼만 아낍니다. 그래서 기존 도시철도가 있으면 공표 표정속도로 시간을 재서 둘 중 빠른 쪽을 기준으로 씁니다.

생략 가능 · 첫 줄 시간가치를 보시면, 지침의 2013년 값에 소비자물가지수를 곱해 2025년으로 환산했습니다. 환산 지수로 CPI 를 쓴 것도 지침 6.2.3 규정입니다. 건설비는 GDP 디플레이터를 쓰는데 계열이 다릅니다.

생략 가능 · 왼쪽 아래 칸입니다. 시간가치는 등급을 맞춰 골라야 합니다. 광역도시철도는 업무 통행이 4퍼센트밖에 안 되기 때문에 비업무 시간가치가 값을 지배합니다. 여기에 고속철도 값을 쓰면 편익이 부풀려집니다.

오른쪽 아래가 최근에 고친 부분입니다. 처음에는 이용객 모두가 노선 끝에서 끝까지 탄다고 봤습니다. 그러면 노선이 길수록 한 사람이 아끼는 시간이 끝없이 커집니다. 서울에서 부산을 그었더니 광역급행이 이미 있는 KTX 보다 한 시간 넘게 느린데도 B/C 가 3.86 이 나왔습니다. 서울시 교통 빅데이터에서 지하철 한 번 이동이 평균 13.6킬로미터라는 값을 찾아, 한 사람은 그만큼만 탄다고 고쳤습니다. 지금 서울~부산 광역급행은 0.15 입니다. 13.6킬로미터만 타는 사람은 역이 수십 킬로미터마다 있는 KTX 를 탈 수 없으니, KTX 는 전환 전 수단에서 뺐습니다.

생략 가능 · 참고로 화면의 건설비는 명목 총액이고 B/C 분모는 현재가치라, 표의 건설비로 나눠도 B/C 가 안 나옵니다. 화면에 각주로 적어 두었습니다.

솔직하게 말씀드리면 편익은 통행시간 절감만 셉니다. 운영비 절감이나 사고 감소, 환경 편익은 아직 안 넣었습니다. 그래서 B/C 절대값은 실제보다 낮게 나옵니다.

12. 추천 알고리즘 — 20개 조합을 걸러 하나를 남깁니다

전문 1.7분 · 줄이면 1.1분

추천 알고리즘입니다. 다섯 단계로 거릅니다.

생략 가능 · 먼저 수단 열 개와 구조 두 개, 스무 조합을 전부 계산합니다. 거르기 전에 다 구하는 게 중요합니다. 사용자가 표에서 전부 볼 수 있어야 하니까요.

생략 가능 · 2단계는 운행 가능성입니다. 첨두 시간 단면 수요가 그 수단의 수송능력을 넘으면 탈락입니다.

3단계는 B/C 가 0 보다 커야 합니다. 이미 있는 노선보다 느린 안은 아끼는 시간이 없어 B/C 가 0 인데, 처음에는 그런 안도 추천에 올라왔습니다. 아무 시간도 아끼지 못하는 안을 내세운 셈입니다. 전부 0 이면 추천할 신설안이 없다고 표 위에 적습니다.

4단계가 오늘 제일 말씀드리고 싶은 부분입니다. 오른쪽 칸을 봐 주십시오.

서울에서 부산을 그었더니 340킬로미터짜리 BRT 가 B/C 7.83 으로 1위였습니다. 버스 전용차로니까 싸거든요. 그런데 부산까지 가는 버스는 현실의 답이 아닙니다.

생략 가능 · 여기서 선택지가 둘이었습니다. 표에서 지우거나, 남기되 추천에서 빼거나. 저는 남겼습니다. 사용자의 선택을 막지 않는 게 이 서비스의 방식이고, 다만 기본으로 내세우지만 않습니다.

그리고 맨 아래 청록색 문장. 이걸 고칠 때 백엔드만 고쳤더니, 지도에는 광역급행이 그려지는데 표에는 BRT 가 굵게 표시되는 일이 생겼습니다. 화면 하나가 두 개의 답을 동시에 가리킨 겁니다. 지금은 시나리오 비교 화면까지 네 곳이 같은 규칙을 씁니다.

13. 정차역 자동 배치 — 속도를 정하는 것은 기술이 아니라 정차 횟수였습니다

전문 2.3분 · 줄이면 1.8분

정차역 자동 배치입니다.

먼저 표정속도가 무엇인지 말씀드리면, 구간 거리를 전체 소요시간으로 나눈 값입니다. 달리는 시간만이 아니라 역마다 서는 시간과 가감속까지 다 들어갑니다. 지하철 전동차 최고속도는 80km/h 인데 표정속도는 32.4km/h 입니다. 차가 느린 게 아니라 계속 서기 때문입니다.

생략 가능 · 맨 위 식이 어떻게 나왔는지 말씀드리면, 운영 중인 노선 18개의 실측 표정속도와 실측 역간격을 모아 최소제곱 회귀를 돌린 결과입니다. 제가 정한 숫자가 아닙니다.

생략 가능 · 12.65 가 기울기입니다. 역간격이 1킬로미터 넓어지면 표정속도가 12.65 붙는다는 뜻입니다. 18.56 은 절편인데, 역간격이 0 인 노선은 없으니 그 자체로는 뜻을 읽지 않습니다.

그래서 그 표정속도가 역간격으로 거의 다 설명됩니다. 결정계수가 0.925 입니다. 역간격이 0.71 에서 2.36 킬로미터 사이면 수단이 트램이든 지하철이든 구분하지 않아도 속도의 93 퍼센트가 설명됩니다.

이게 뜻하는 바가 중요합니다. 속도를 정하는 건 차량 기술이 아니라 몇 번 서느냐입니다. 그래서 정차역을 제대로 놓는 게 B/C 에 직결됩니다.

아래 붉은 칸이 제가 이번에 고친 버그입니다.

서울에서 부산까지 318킬로미터를 그었는데 역이 9개만 놓였습니다. 반경 500미터에 인구가 없으면 역을 빼는 규칙 때문이었습니다. 강이나 호수 위를 막으려고 넣은 규칙인데요.

원인을 보니, 집계구 자료 자체가 역 주변으로 모은 것이었습니다. 그러니까 아직 자료를 안 모은 시골 구간이 전부 "사람이 없는 곳"으로 읽힌 겁니다.

인구가 0인 것과 인구를 모르는 것은 다릅니다. 그래서 반경 10킬로미터 안에 집계구가 하나라도 있을 때만 — 즉 그 지역 자료를 모은 적이 있을 때만 — 역을 빼도록 고쳤습니다. 9개가 32개가 됐습니다.

14. 검증 — 테스트는 통과시키려고 쓰는 게 아닙니다

전문 2.9분 · 줄이면 2.0분

검증입니다. 위에 숫자가 셋 있는데 단위가 다릅니다. 앞의 둘은 테스트 개수입니다. 백엔드 116개, ETL 260개이고 전부 통과합니다. 오른쪽 1.25와 1.75배는 테스트 개수가 아니라 모델의 예측 오차입니다. 비용 모델이 1.25배, 수요 모델이 1.75배입니다. 표본을 하나씩 빼면서 맞춰 보는 교차검증으로 쟀고, 숨기지 않고 화면에 그대로 적습니다. 예를 들어 건설비가 1조로 나오면 8천억에서 1조 2500억 사이로 읽어 달라는 뜻입니다.

다만 개수보다 어떻게 쓰는지를 말씀드리고 싶습니다. 제목처럼, 테스트는 통과시키려고 쓰는 게 아닙니다. 지키려고 쓰는 겁니다.

생략 가능 · 가운데 칸의 파서는 원자료 파일을 읽어 표로 바꾸는 코드입니다. 공공데이터는 사람이 손으로 적은 거라 같은 뜻을 제각각 씁니다. 물결표만 해도 세 가지 글자가 섞여 있고, 연도가 두 자리로 적힌 칸도 있습니다. 24를 1924년으로 읽으면 물가 환산이 백 년 어긋납니다. 그래서 계산보다 먼저 읽는 코드를 테스트했습니다. 숫자가 아닌 칸을 0으로 채우지 않는 것도 여기서 지킵니다. 0은 값이 0이라는 뜻이고 비어 있는 건 모른다는 뜻이라서요.

왼쪽 아래 칸입니다. 테스트를 쓰고 통과하면 끝이라고 생각했는데, 그게 아니었습니다. 규칙을 일부러 지워 보고 그 테스트만 깨지는지 확인합니다. 안 깨지면 그 테스트는 아무것도 지키지 않고 있던 겁니다. 통과하고 있었을 뿐입니다. 얼마 전에도 그런 일이 있었습니다. KTX 계통을 가를 때 한두 편짜리를 계통으로 부르지 않는 규칙을 넣고 테스트를 썼는데, 규칙을 지워도 테스트가 통과했습니다. 제가 쓴 예가 시간 범위까지 겹쳐서 다른 규칙에 먼저 걸렸던 겁니다. 실제 자료에서 겹치지 않으면서 한 편짜리인 구간을 찾아 예를 바꾸니, 그때부터 규칙을 지우면 깨졌습니다. 꺼 보지 않았으면 못인 줄 알았을 겁니다.

생략 가능 · 가운데 아래 칸. 설명서에 적은 숫자가 코드나 자료와 다르면 테스트가 깨지게 해 뒀습니다. 실제로 설명서에 테스트가 56개라고 적혀 있었는데 세어 보니 62개였습니다. 이런 걸 잡으려고 만든 장치입니다.

오른쪽 칸. 화면은 직접 열어 봐야 합니다. 빌드가 통과하고 API 응답도 정상인데 화면이 깨진 경우가 여러 번 있었습니다. 지도 캔버스 크기가 잘못 잡혀서 빈 곳을 클릭하면 좌표가 어긋난 적도 있습니다.

15. 현재 상태 — 제안서 기능 8개를 모두 구현했습니다

전문 4.7분 · 줄이면 4.1분

현재 상태입니다. 제안서에 적은 기능 여덟 개를 모두 구현했고, 전 구간 실행 검증까지 마쳤습니다.

왼쪽 네 번째 줄, 비워 뒀던 KTX 소요시간 칸을 TAGO 열차정보로 받아 넣었습니다. 서울에서 부산이면 164분이 표에 뜨고, 그 아래 설명 줄에 44편 143에서 176분, 경로가 다른 11편 180에서 205분이 붙습니다. 칸 하나에 네 숫자를 넣으면 아무도 안 읽으니까, 칸에는 빠른 계통의 중앙값 하나만 넣었습니다.

다섯 번째 줄은 그 칸을 채우고 나서 드러난 문제를 고친 것입니다. 표에 KTX 164분이 들어가자, 그보다 한 시간 넘게 느린 광역급행이 B/C 3.86 으로 나오는 게 한 표 안에서 보였습니다. 편익을 이용객 전원이 끝까지 탄다고 셌기 때문입니다. 지금은 기존 도시철도를 전환 전 수단에 넣고, 한 사람은 평균 13.6킬로미터만 탄다고 봅니다. 서울~부산 광역급행은 0.15 가 됐습니다.

오른쪽 남은 것 목록은 숨기지 않고 그대로 적었습니다. 맨 위 줄이 가장 큰 한계입니다. 왼쪽 첫 줄부터 말씀드리면, 호수나 저수지를 지나면 이제 돌아가는 선형을 제안합니다. OpenStreetMap 에서 전국 호수 120개의 경계를 받아, 물을 피해 가는 가장 짧은 길을 찾습니다. 바꾸는 건 사용자가 버튼을 눌렀을 때뿐입니다. 다만 돌아가는 길이 원래의 1.5배를 넘으면 다리로 건너는 편이 낫다고 보고 제안하지 않는데, 이 1.5배는 가정값입니다. 교량 단가 자료가 없어 다리와 우회를 견줄 수 없기 때문입니다. 강은 돌아갈 수 없으니 그대로 건너되, 노선이 물 위를 지나는 길이를 재서 지도에 파랗게 칠하고 "한강 0.8킬로미터, 교량 할증은 넣지 않았다" 라고 적습니다. 한강 같은 강 조각 천 개를 같은 방법으로 받았습니다. 할증 자체는 단가가 없어 여전히 못 넣습니다.

두 번째 줄 BRT 표정속도는 세종시 버스 위치를 직접 받아 재는 것까지 끝냈습니다. 그런데 미리 정해 둔 네 조건 중 대표성 하나를 못 넘었습니다. 같은 방법으로 잰 세종 시내버스가 어느 시간대에도 공표 연평균 21.6 까지 내려오지 않아서, 측정값을 연평균으로 옮길 근거가 없습니다. 더 모아서 풀리는 문제가 아니라 재는 방식이 다른 문제라, 가정값을 그대로 두고 그 이유를 적었습니다.

생략 가능 · 세 번째 줄 지하비율은 비용 모델에서 말씀드렸듯 17건을 실측으로 바꿨고, 나머지 23건은 아직 공사 중이라 잴 수 없습니다.

마지막 줄 운영비는 최근에 따로 조사했습니다. 지금 비용에는 건설비만 들어가서 B/C 가 실제보다 높게 나옵니다. KDI 예비타당성조사 보고서 아홉 건을 읽어 보니, 연간 운영비가 총사업비의 지하철 2% 안팎, 경전철 1.6% 안팎이었고, 넣으면 B/C 가 0.75 에서 0.79배 정도로 내려갑니다. 그런데 복선전철은 옛 지침으로 계산한 사례뿐이고 트램은 예타 자체가 없어서, 수단마다 같은 근거로 넣을 수가 없었습니다. 일부 수단에만 넣으면 그 수단만 불리해져 비교가 기울기 때문에, 계산에는 넣지 않고 화면의 B/C 옆에 "운영비를 넣지 않아 높게 나온다" 고 적어 두었습니다.

마지막 칸이 제가 이 프로젝트에서 가장 많이 배운 것입니다. 처음에는 값을 못 구하면 그럴듯한 숫자로 채우고 싶었습니다. 그런데 한 번 적히면 다음 사람은 그걸 공표값으로 믿습니다. 출처를 의심하지 않게 되는 거죠. 그래서 규칙을 세웠습니다. 못 채운 칸은 비우고 이유를 적는다. 화면에도 적고 문서에도 적는다.

생략 가능 · KTX 소요시간이 이 규칙을 두 번 시험했습니다. 처음에는 자료가 없다고 적었는데 사실은 제가 안 받은 것이었습니다. 받고 나서도 두 군데가 비어 있었는데 그것도 자료가 없는 게 아니라 제가 등급을 하나만 받고 이름을 못 맞춘 것이었습니다. 자료가 없다와 내가 안 받았다는 다른 말이라는 걸, 같은 칸에서 세 번 배웠습니다.

이상으로 발표를 마치겠습니다. 감사합니다.